

openssl-bruteforce

The purpose of this program is to try to find the password of a file that was encrypted with the 'openssl' command (e.g.: `openssl enc -aes256 -salt -in clear.file -out encrypted.file`).

It can be used in two ways:

- try all the possible passwords given a charset
- try all the passwords in a file

There is a command line option to specify the number of threads to use.

The program should be able to use all the digests and symmetric ciphers available with the OpenSSL libraries installed on your system.

Sending a USR1 signal to a running bruteforce-salted-openssl process makes it print progress and continue.

Exhaustive mode

The program tries to decrypt the file by trying all the possible passwords. It is especially useful if you know something about the password (i.e. you forgot a part of your password but still remember most of it). Finding the password of the file without knowing anything about it would take way too much time (unless the password is really short and/or weak).

There are command line options to specify:

- the minimum password length to try
- the maximum password length to try
- the beginning of the password
- the end of the password
- the character set to use (among the characters of the current locale)

Dictionary mode

The program tries to decrypt the file by trying all the passwords contained in a file. The file must have one password per line.

Dependencies

The program requires the OpenSSL libraries.

Available options

- | | |
|------------------------------|---|
| <code>-1</code> | Stop the program after finding the first password candidate. |
| <code>-a</code> | List the available cipher and digest algorithms. |
| <code>-B <file></code> | Search using binary passwords (instead of character passwords). |

```
Write candidates to <file>.

-b <string> Beginning of the password.
             default: ""

-c <cipher>  Cipher for decryption.
             default: aes-256-cbc

-D          Display salt, key and iv along with the tested password.

-d <digest> Digest for key and initialization vector generation.
             default: sha256

-e <string> End of the password.
             default: ""

-f <file>    Read the passwords from a file instead of generating them.

-h          Show help and quit.

-i <n>       With -K, set the PBKDF2 iteration count to <n>.
             default: 10000

-K          Use PKCS5_PBKDF2_HMAC to derive the key.

-L <n>       Limit the maximum number of tested passwords to <n>.

-l <length> Minimum password length (beginning and end included).
             default: 1

-M <string> Consider the decryption as successful when the data starts
considered  with <string>. Without this option, the decryption is
            as successful when the data contains mostly printable ASCII
            characters (at least 90%).

-m <length> Maximum password length (beginning and end included).
             default: 8

-N          Ignore decryption errors (similar to openssl -nopad).

-n          Ignore salt (similar to openssl -nosalt).

-p <n>       Preview and check the first <n> decrypted bytes for the
magic       string. If the magic string is present, try decrypting the
rest        of the data.
             default: 1024

-s <string> Password character set.
             default: "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ
                    VWXYZabcdefghijklmnopqrstuvwxyz"
```

<code>-t <n></code>	Number of threads to use. default: 1
<code>-v <n></code>	Print progress info every n seconds.
<code>-w <file></code>	Restore the state of a previous session if the file exists, then write the state to the file regularly (~ every minute).

Limitations

The program considers decrypted data as correct if it is mainly composed of printable ASCII characters (at least 90%). If the file you want to decrypt doesn't contain plain text, you will have to either use the `-M` option, or modify the 'valid_data' function in the source code to match your needs.

If the file you want to decrypt is big, you should use the `-N` option on a truncated version of the file (to avoid decrypting the whole file with each password).

Examples

Try to find the password of an aes256 encrypted file using 4 threads, trying only passwords with 5 characters:

```
bruteforce-salted-openssl -t 4 -l 5 -m 5 -c aes256 encrypted.file
```

Try to find the password of a des3 encrypted file using 8 threads, trying only passwords with 9 to 11 characters, beginning with "AbCD", ending with "Ef", and containing only letters:

```
bruteforce-salted-openssl -t 8 -l 9 -m 11 -c des3 -b "AbCD" -e "Ef" \  
-s "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz" encrypted.file
```

Try to find the password of an aes256 encrypted file using 6 threads, trying the passwords contained in a dictionary file:

```
bruteforce-salted-openssl -t 6 -f dictionary.txt -c aes256 encrypted-file
```

Print progress info every 30 seconds:

```
bruteforce-salted-openssl -t 6 -f dictionary.txt -c aes256 -v 30 encrypted-  
file
```

Try to find the password of a des3 encrypted gzip file using 8 threads:

```
bruteforce-salted-openssl -t 8 -v 5 -c des3 -m 9 -s "0123456789" -M "$(echo -ne '\x1f\x8b')" file.tar.gz.des3
```

Save/restore state between sessions:

```
bruteforce-salted-openssl -t 6 -f dictionary.txt -c aes256 -w state.txt encrypted-file  
(Let the program run for a few minutes and stop it)  
bruteforce-salted-openssl -t 6 -c aes256 -w state.txt encrypted-file
```

Show the list of available algorithms:

```
bruteforce-salted-openssl -a
```

If the program finds a candidate password 'pwd', you can decrypt the data using the 'openssl' command:

```
openssl enc -d -aes256 -salt -in encrypted.file -out decrypted.file -k pwd
```